

Reviewer Pack — Schur-Weyl Duality Invariant for Permanent vs Determinant Ci...

Entry 0032e472e3ec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 12:38:06 UTC

Schur-Weyl Duality Invariant for Permanent vs Determinant Circuit Complexity

Entry ID: 0032e472e3ec **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 12:38:06 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Boolean Circuit Complexity (Permanent vs Determinant)

Statement:

For a homogeneous polynomial f of degree n in n variables, the number of standard Young tableaux in its decomposition under $GL(n)$ is exponential in n for the permanent and polynomial in n for the determinant.

Rationale (proposer's reasoning):

Schur-Weyl duality reveals structural differences in the representation theory of the permanent and determinant, which may imply inherent complexity gaps. The number of tableaux could reflect the number of terms in the decomposition, directly relating to circuit size.

Taxonomy category: GCT_DET_PERM (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 96d9adcd5123444f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=103.9s

5.1 Generated Python source

```
import random
import math
from itertools import combinations, permutations

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    result = [[0]*n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0]*n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j]*x[j] for j in range(i+1, n))) / A[i][i]
    return x
```

```

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    det = 0
    for c in range(n):
        submatrix = [row[:c] + row[c+1:] for row in matrix[1:]]
        sign = (-1) ** (c % 2)
        sub_det = determinant(submatrix)
        det += sign * matrix[0][c] * sub_det
    return det

def permanent(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    perm = list(permutations(range(n)))
    det = 0
    for p in perm:
        prod = 1
        for i in range(n):
            prod *= matrix[i][p[i]]
        det += prod
    return det

def young_tableaux_count(n, k):
    if n == 0 or k == 0:
        return 1
    return sum(young_tableaux_count(i, k-1) * young_tableaux_count(n-i-1, k-1) for i in range(k))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    poly = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    perm_count = permanent(poly)
    det_count = determinant(poly)
    tableaux_count = young_tableaux_count(n, n)

    conjecture_holds = perm_count > 2**n and det_count <= n**n
    counterexample = "" if conjecture_holds else "Permanent is not exponential or determinant is not polynomial"

    return {
        "metric_name": "tableaux_count",
        "metric_value": tableaux_count,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:

```

```

    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Permanent is not exponential or determinant is\")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing data, preventing evaluation of support fraction or counterexamples. | next: Re-run test with increased resource limits and debug logging to identify crash cause

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106902
2	preregistration	ollama_remote	qwen3:8b	0	0	24209
3	novelty	ollama_remote	qwen3:8b	0	0	20681
4	novelty	ollama_remote	qwen3:8b	0	0	16921
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21123
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12094
7	judge	ollama_remote	qwen3:8b	0	0	132107

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 334037 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/0032e472e3ec.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0032e472e3ec.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0032e472e3ec.tar.gz` (if generated)